CENG 111 Algoritmalar ve Programlama 8: İşaretçiler

Öğr.Gör. Şevket Umut ÇAKIR

Pamukkale Üniversitesi

Hafta 8

Anahat

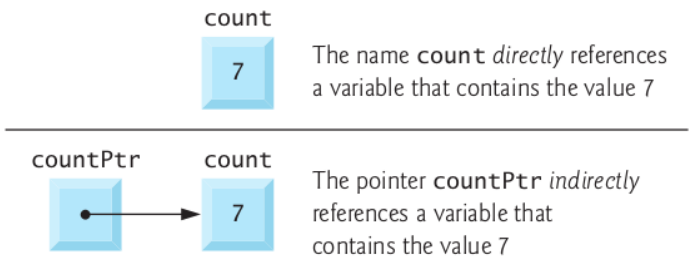
1. Giriş
2. İşaretçi Değişken Tanımları ve Başlatma
3. İşaretçi Operatörleri
4. Fonksiyonlara Argümanları Referans Olarak Geçme
5. İşaretçilerde const Kullanmak
6. Referans Olarak Geçme ile Kabarcık Sıralama
7. sizeof Operatörü
8. İşaretçi İfadeleri ve Aritmetiği
9. Diziler ve İşaretçiler Arasındaki İlişki
10. İşaretçi Dizileri
11. Rastgele Sayı Vaka Analizi
12. Fonksiyon İşaretçileri

Giriş

- İşaretçiler, diğer değişkenlerin bellek adreslerini depolayan değişkenlerdir.
- Bir işaretçi kullanarak bir değere başvurmaya "dolaylı erişim" denir.
- İşaretçiler kullanılmadan önce tanımlanmalıdır. Bir işaretçi tanımındaki $*$ sembolü, değişkenin bir işaretçi olduğunu gösterir.

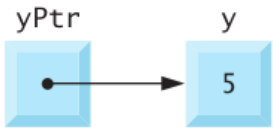
İşaretçi Değişken Tanımları ve Başlatma

- İşaretçi bildirimi `int *countPtr;` biçiminde yapılabilir.
- İşaretçiler `NULL`, 0 veya bir bellek adresi ile başlatılabilir.
- Değeri `NULL` olan bir işaretçi hiçbir şeyi göstermez.
- NULL, 0 değerine sahip sembolik bir sabittir ve `<stddef.h>` başlık dosyasında tanımlanmıştır.
- `int count=7; int *countPtr=&count;` ifadesinin bellekteki yerleşimi aşağıda gösterilmiştir.



Adres Operatörü(&)

- Adres Operatörü(&), bir değişkenin adresini verir
- `int y = 5; int *yPtr = &y;`



- Bellekteki gösterimi:



Dolaylılık Operatörü(*)

- Dolaylılık operatörü(*), işaretçinin göstediği değeri almaya yarar
- `printf("%d", *yPtr);` ifadesi çalıştırıldığında ekrana 5 yazar.

İşaretçi Operatörleri

```c
// fig07_01.c
// & ve * işaretci operatorlerinin kullanımı
#include <stdio.h>

int main(void) {
   int a = 7;
   int *aPtr = &a; // aPtr'ye a'nın adresini ata

   printf("a'nın adresi: %p\naPtr'nin değeri: %p\n\n", &a, aPtr);
   printf("a'nın değeri: %d\n*aPtr'nin değeri:%d\n\n", a, *aPtr);
   printf("* ve & birbirinin tam tersi\n");
   printf("&*aPtr = %p\n*&aPtr = %p\n", &*aPtr, *&aPtr);
}
```

İşaretçi Operatörleri

```
a'nın adresi: 0x7fffffffd22c
aPtr'nin değeri: 0x7fffffffd22c

a'nın değeri: 7
*aPtr'nin değeri:7

* ve & birbirinin tam tersi
&*aPtr = 0x7fffffffd22c
*&aPtr = 0x7fffffffd22c
```

İşaretçi Operatörleri

- "%p", bellek konumlarını onaltılık tabanda(hexadecimal) yazdırmayı sağlar

İşaretçi Operatörleri

Öncelik

Operators	Grouping	Type
() [] ++ *(postfix)* -- *(postfix)*	left to right	postfix
+ - ++ -- ! * & (*type*)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > s>=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Fonksiyonlara Argümanları Referans Olarak Geçme

- İki türde değer geçme mekanizması bulunur:
  - Değer olarak geçme(pass by value)
  - Referans olarak geçme(pass by reference)
- $*$ ve $\&$ operatörleri referans olarak geçme için kulllanılır.

Değer Olarak Geçme

```c
// fig07_02.c
// Değer olarak geçme ile küp hesaplaması.
#include <stdio.h>

int kupDegerOlarak(int n); // prototip

int main(void) {
   int sayi = 5; // sayıyı başlat

   printf("Sayının original değeri %d", sayi);
   sayi = kupDegerOlarak(sayi); // pass number by value to cubeByValue
   printf("\nSayının yeni değeri %d\n", sayi);
}

// küpü deger olarak hesapla ve sonucu geri döndür
int kupDegerOlarak(int n) {
   return n * n * n; // yerel değişkenin küpünü al ve geri döndür
}
```

Değer Olarak Geçme

```
Sayının original değeri 5
Sayının yeni değeri 125
```

Referans Olarak Geçme

```c
// fig07_03.c
// Referans olarak geçme ile küp hesaplaması.

#include <stdio.h>

void kupReferansOlarak(int *nPtr); // fonksiyon prototipi

int main(void) {
   int sayi = 5; // sayıyı başlat

   printf("Sayının original değeri %d", sayi);
   kupReferansOlarak(&sayi); // kupReferansOlarak ile adresi gönder
   printf("\nSayının yeni değeri %d\n", sayi);
}

// *nPtr'nin küpünü hesapla; mainde de değişir
void kupReferansOlarak(int *nPtr) {
   *nPtr = *nPtr * *nPtr * *nPtr; // *nPtr küpünü al
}
```

Referans Olarak Geçme

```
Sayının original değeri 5
Sayının yeni değeri 125
```

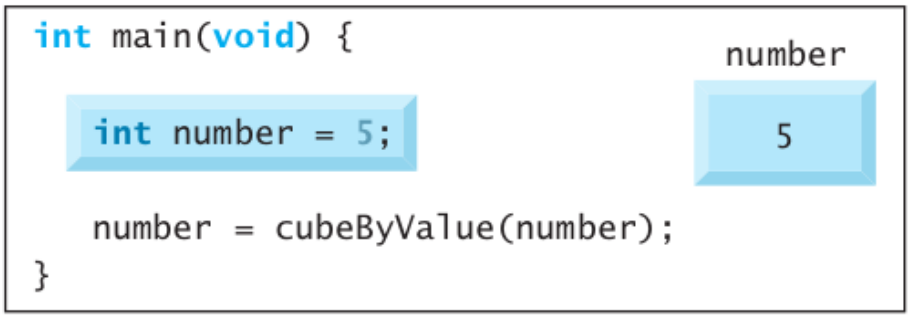
Referans Olarak Geçme

- Fonksiyon prototipinde `int *` ifadesinden sonra değişken adı opsiyoneldir.
- Bir fonksiyona tek boyutlu bir dizi göndermek için işaretçi türü kullanılabilir
  - `void kupReferansOlarak(int *nPtr)`
  - Dizi gösterimi ile arasında bir fark yoktur, derleyici otomatik dönüştürür
-

Değer Olarak Geçme

Adım 1

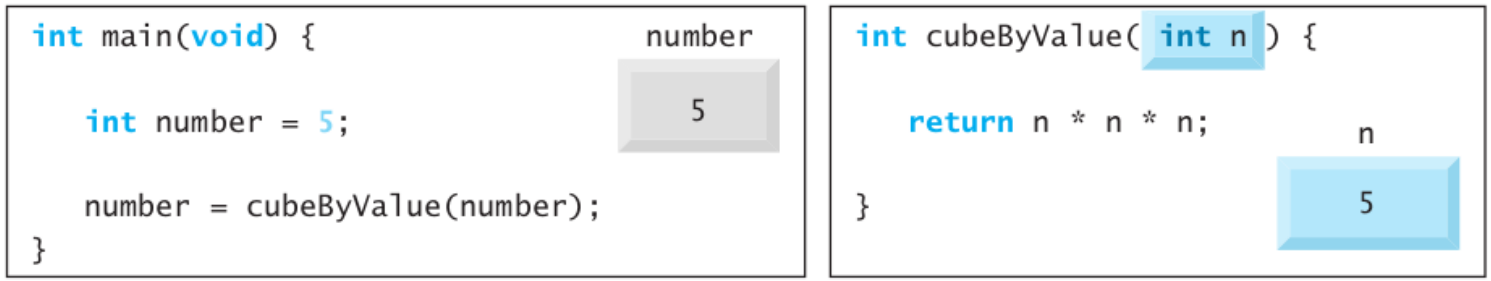
Step 1: Before `main` calls `cubeByValue`:



Değer Olarak Geçme

Adım 2

Step 2: After **cubeByValue** receives the call:



Değer Olarak Geçme

Adım 3

Step 3: After **cubeByValue** cubes parameter **n** and before **cubeByValue** returns to **main**:

```
int main(void) {

   int number = 5;

   number = cubeByValue(number);
}
```

number: 5

```
int cubeByValue(int n) {

   return n * n * n;

}
```

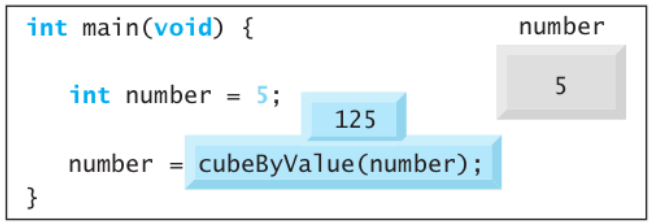
125

n: 5

Değer Olarak Geçme

Adım 4

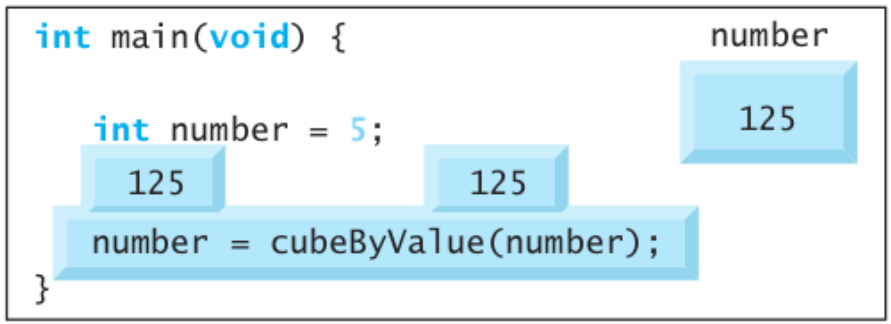
Step 4: After **cubeByValue** returns to **main** and before assigning the result to **number**:



Değer Olarak Geçme

Adım 5

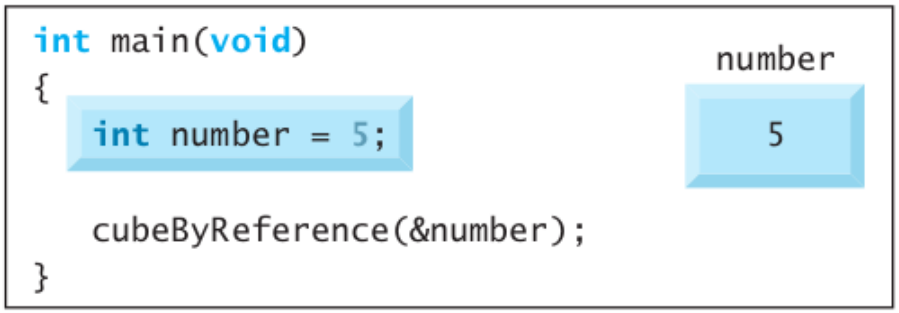
Step 5: After `main` completes the assignment to `number`:



Referans Olarak Geçme

Adım 1

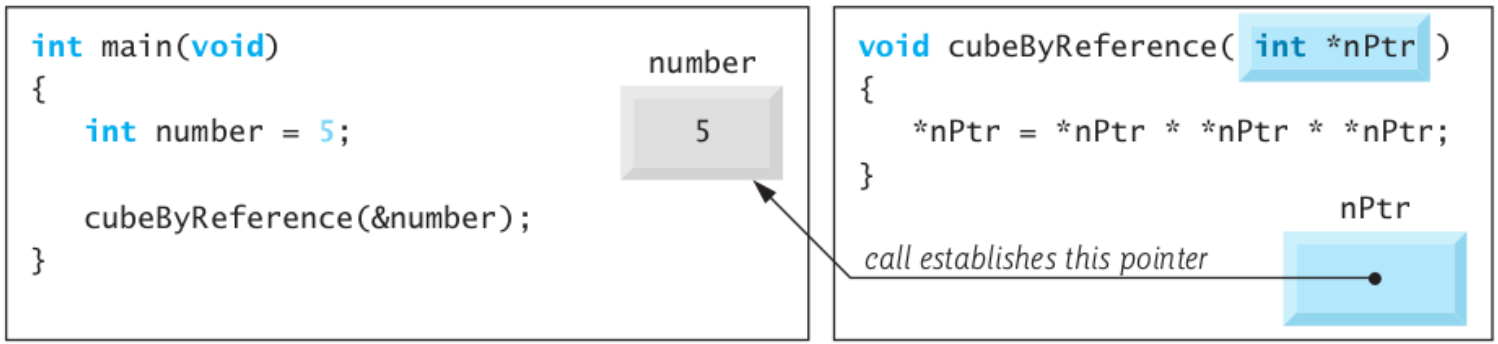
Step 1: Before `main` calls `cubeByReference`:



Referans Olarak Geçme

Adım 2

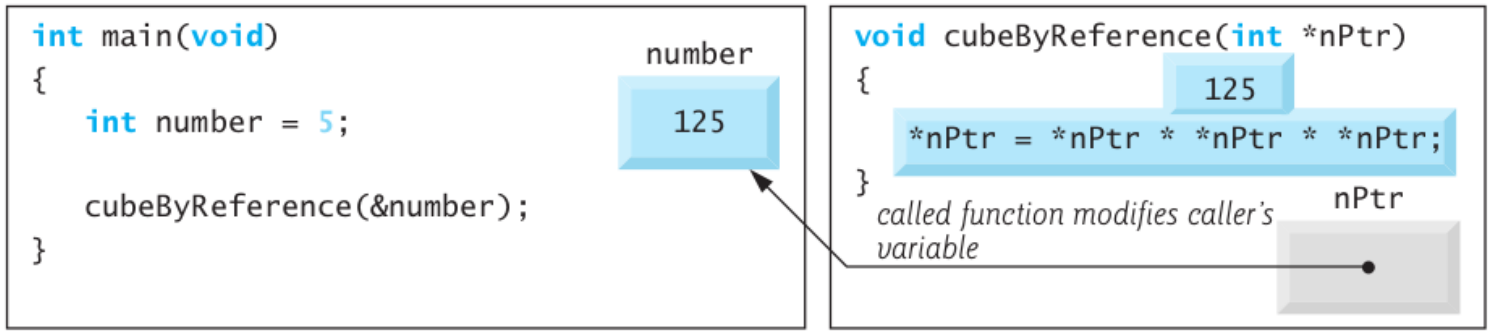
Step 2: After **cubeByReference** receives the call and before ***nPtr** is cubed:



Referans Olarak Geçme

Adım 3

Step 3: After *nPtr is cubed and before program control returns to main:



İşaretçilerde const Kullanmak

- `const` anahtar sözcüğü, ilgili değişkenin değerinin değiştirilemeyeceğini derleyiciye bildirir
- Bu tür değişkenler **sabit** olarak isimlendirilir
- Bir fonksiyona veri işaretçisi göndermenin dört yolu vardır:
  - Sabit olmayan verilere yönelik sabit olmayan bir işaretçi.
  - Sabit olmayan verilere yönelik sabit bir işaretçi.
  - Sabit verilere yönelik sabit olmayan bir işaretçi.
  - Sabit verilere yönelik sabit bir işaretçi.
- Bu durumlar örneklerle açıklanmaya çalışılacaktır

String Büyük Harfe Dönüştürme

Sabit olmayan verilere yönelik sabit olmayan bir işaretçi.

```
4  #include <ctype.h>
5  #include <stdio.h>
6
7  void buyukHarfeDonustur(char *sPtr); // prototype
8
9  int main(void) {
10    char string[] = "kArAkTerLer ve 32.98 Tl"; // karakter dizisini başlat
11
12    printf("Dönüşüm öncesi string: %s\n", string);
13    buyukHarfeDonustur(string);
14    printf("Dönüşüm sonrası string: %s\n", string);
15 }
16
17 // convert string to uppercase letters
18 void buyukHarfeDonustur(char *sPtr) {
19    while (*sPtr != '\0') { // mevcut karakter '\0' olana kadar
20       *sPtr = toupper(*sPtr); // büyük harfe dönüştür
21       ++sPtr; // sPtr'yi sonraki karaktere yönlendir
22    }
23 }
```

String Büyük Harfe Dönüştürme

Ekran Çıktısı

```
Dönüşüm öncesi string: kArAkTerLer ve 32.98 Tl
Dönüşüm sonrası string: KARAKTERLER VE 32.98 TL
```

- `toupper` fonksiyonunu kullanmak için `ctype.h` başlık dosyası eklenmelidir
- Türkçe karakter dönüşümlerinde(Non-ASCII) sorun olabilir

String Yazdırma

Sabit olmayan verilere yönelik sabit bir işaretçi.

```
5  #include <stdio.h>
6  void karakterleriYazdir(const char *sPtr);
7  int main(void) {
8     // initialize char array
9     char string[] = "bir dizenin karakterlerini yazdirmak";
10
11    puts("Metin:");
12    karakterleriYazdir(string);
13    puts("");
14 }
15
16 // sPtr, işaret ettiği karakteri değiştirmek için kullanılamaz,
17 // yani, sPtr "salt okunur" bir işaretçidir
18 void karakterleriYazdir(const char *sPtr) {
19    // tüm dizge boyunca döngü
20    for (; *sPtr != '\0'; ++sPtr) { // başlatma yok
21       printf("%c", *sPtr);
22    }
23 }
```

Sabit Veriyi Değiştirmeye Çalışmak

```
1  // fig07_08.c
2  // Sabit verilere sabit olmayan bir işaretçi aracılığıyla
3  // verileri değiştirmeye çalışılıyor.
4  #include <stdio.h>
5  void f(const int *xPtr); // prototype
6
7  int main(void) {
8     int y = 7; // y'yi tanımla
9
10    f(&y); // f geçersiz değişiklik girişiminde bulunur
11 }
12
13 // xPtr, işaret ettiği değişkenin
14 // değerini değiştirmek için kullanılamaz
15 void f(const int *xPtr) {
16    *xPtr = 100; // hata: const bir nesne değiştirilemez
17 }
```

Sabit Olmayan Veriye Sabit İşaretçiyi Değiştirmek

```c
// fig07_09.c
// Sabit olmayan verilere ait sabit bir işaretçiyi değiştirme girişimi.
#include <stdio.h>

int main(void) {
   int x = 0; // x'i tanımla
   int y = 0; // y'i tanımla

   // ptr, değiştirilebilen bir tam sayıya sabit bir işaretçidir
   // ptr aracılığıyla, ancak ptr her zaman aynı bellek konumunu işaret eder
   int * const ptr = &x;

   *ptr = 7; // izin verilir: *ptr sabit değil
   ptr = &y; // hata: ptr sabittir; yeni adres atanamaz
}
```

Kabarcık Sıralama

Referans Kullanarak

```
4  #include <stdio.h>
5  #define BOYUT 10
6
7  void kabarcikSirama(int * const dizi, size_t boyut); // prototip
8  void degistir(int *eleman1Ptr, int *eleman2Ptr); // prototip
9  int main(void) {
10    // dizi a'yı başlat
11    int a[BOYUT] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
12    puts("Veri ögeleri orijinal sırada");
13    // dizi a boyunca döngü
14    for (size_t i = 0; i < BOYUT; ++i) {
15       printf("%4d", a[i]);
16    }
17    kabarcikSirama(a, BOYUT); // diziyi sırala
18    puts("\nVeri ögeleri artan sırada");
19    // dizi a boyunca döngü
20    for (size_t i = 0; i < BOYUT; ++i) {
21       printf("%4d", a[i]);
22    }
23    puts("");
24 }
```

Kabarcık Sıralama

Referans Kullanarak

```
26 // bir tamsayı dizisini kabarcık sıralama algoritmasıyla sırala
27 void kabarcikSirama(int * const dizi, size_t boyut) {
28    // geçişleri kontrol eden döngü
29    for (int gecis = 0; gecis < boyut - 1; ++gecis) {
30       // her geçiş sırasında karşılaştırmaları kontrol eden döngü
31       for (size_t j = 0; j < boyut - 1; ++j) {
32          // bitişik elemanlar sırasızsa yer değiştir
33          if (dizi[j] > dizi[j + 1]) {
34             degistir(&dizi[j], &dizi[j + 1]);
35          }
36       }
37    }
38 }
```

Kabarcık Sıralama

Referans Kullanarak

```
40 // element1Ptr ve element2Ptr'nin işaret ettiği bellek
41 // konumlarındaki değerleri değiştir
42 void degistir(int *eleman1Ptr, int *eleman2Ptr) {
43    int tut = *eleman1Ptr;
44    *eleman1Ptr = *eleman2Ptr;
45    *eleman2Ptr = tut;
46 }
```

```
Veri ögeleri orijinal sırada
   2   6   4   8  10  12  89  68  45  37
Veri ögeleri artan sırada
   2   4   6   8  10  12  37  45  68  89
```

Kabarcık Sıralama

- `degistir` fonksiyonu iki değişkenin değerini değiştirir
  - Dizinin diğer elemanlarına erişimi yoktur
- `degistir(&dizi[j], &dizi[j + 1]);` ifadesi ile yer değiştirilecek iki komşunun adresi fonksiyona gönderilir
- Sıralama fonksiyonu prototipi:
  ```
  void kabarcikSirama(int * const dizi, size_t boyut);
  ```
  - `int dizi[]` biçiminde de olabilirdi
  - `boyut` parametresi dizi boyutunu saklar, bu bilgi dizi değişkeninde bulunmamaktadır
- C dilinde bir fonksiyona dizi türünde bir değişken gönderiliyorsa, boyutu da belirtilmelidir

sizeof Operatörü

- C dili, herhangi bir nesnenin ya da türün boyutunu **bayt** cinsinden belirlemek için `sizeof` operatörünü sağlar.
- Bir dizi ismine uygulandığında(sonraki slayt), `size_t` türünde, dizinin kaç bayt yer kapladığını belirtir
  - `size_t`, en az 16 bitlik işaretsiz bir tamsayıdır.

sizeof Operatörü

```c
// fig07_12.c
// Bir dizi adına sizeof uygulanması, dizideki bayt sayısını döndürür.
#include <stdio.h>
#define BOYUT 20

size_t boyutBul(const float *ptr); // prototip

int main(void){
   float dizi[BOYUT]; // dizi oluştur

   printf("Dizideki bayt sayısı %zu\n", sizeof(dizi));
   printf("boyutBul tarafından döndürülen bayt sayısı %zu\n",
   ↪  boyutBul(dizi));
}

// ptr'nin boyutunu döndür
size_t boyutBul(const float *ptr) {
   return sizeof(ptr);
}
```

sizeof Operatörü

```
Dizideki bayt sayısı 80
boyutBul tarafından döndürülen bayt sayısı 8
```

- Eski 32 bit sistemlerde `boyutBul` **4** verir
- `double real[22];` ifadesini düşünelim

sizeof Operatörü

```
Dizideki bayt sayısı 80
boyutBul tarafından döndürülen bayt sayısı 8
```

- Eski 32 bit sistemlerde `boyutBul` **4** verir
- `double real[22];` ifadesini düşünelim
  - Bir `double`, 8 bayt alan kaplar. `sizeof(real)`, 176 verir.
- `sizeof(real) / sizeof(real[0])` ifadesi, dizide kaç eleman olduğunu verir.

sizeof Operatörü

```c
// fig07_13.c
// sizeof operatörünü kullanarak standart türlerin boyutlarının belirlenmesi
#include <stdio.h>

int main(void) {
   char c = '\0';
   short s = 0;
   int i = 0;
   long l = 0;
   long long ll = 0;
   float f = 0.0F;
   double d = 0.0;
   long double ld = 0.0;
   int array[20] = {0}; // 20 elemanlı bir tamsayı dizisi oluştur
   int *ptr = array; // diziye bir işaretçi oluştur

```

sizeof Operatörü

```
16
17      printf("    sizeof c = %2zu\t       sizeof(char) = %2zu\n",
18          sizeof c, sizeof(char));
19      printf("    sizeof s = %2zu\t      sizeof(short) = %2zu\n",
20          sizeof s, sizeof(short));
21      printf("    sizeof i = %2zu\t        sizeof(int) = %2zu\n",
22          sizeof i, sizeof(int));
23      printf("    sizeof l = %2zu\t       sizeof(long) = %2zu\n",
24          sizeof l, sizeof(long));
25      printf("   sizeof ll = %2zu\t  sizeof(long long) = %2zu\n",
26          sizeof ll, sizeof(long long));
27      printf("    sizeof f = %2zu\t      sizeof(float) = %2zu\n",
28          sizeof f, sizeof(float));
29      printf("    sizeof d = %2zu\t     sizeof(double) = %2zu\n",
30          sizeof d, sizeof(double));
31      printf("   sizeof ld = %2zu\tsizeof(long double) = %2zu\n",
32          sizeof ld, sizeof(long double));
33      printf("sizeof array = %2zu\n  sizeof ptr = %2zu\n",
34          sizeof array, sizeof ptr);
35  }
```

sizeof Operatörü

```
    sizeof c =  1          sizeof(char) =  1
    sizeof s =  2         sizeof(short) =  2
    sizeof i =  4           sizeof(int) =  4
    sizeof l =  8          sizeof(long) =  8
   sizeof ll =  8     sizeof(long long) =  8
    sizeof f =  4          sizeof(float) =  4
    sizeof d =  8         sizeof(double) =  8
   sizeof ld = 16   sizeof(long double) = 16
sizeof array = 80
  sizeof ptr =  8
```

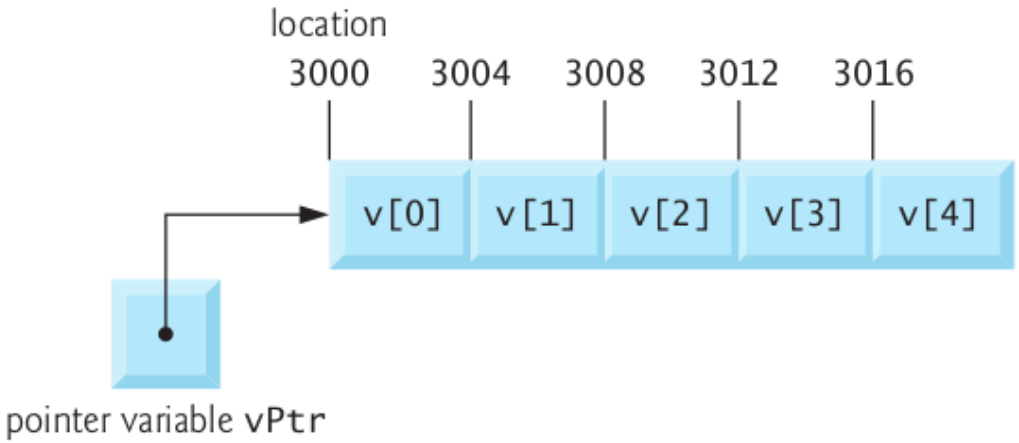
İşaretçi İfadeleri ve Aritmetiği

- İşaretçiler üzerinde aşağıdaki operatörler kullanılabilir
  - Artırma (++) ya da azaltma (--)
  - Bir işaretçiye tamsayı ekleme(+ ya da +=)
  - Bir işaretçiden bir tamsayı çıkarma(- ya da -=)
  - Bir işaretçiden başka işaretçiyi çıkarma
    - Ancak iki işaretçi de aynı diziyi gösterirse anlamlı

İşaretçi İfadeleri ve Aritmetiği

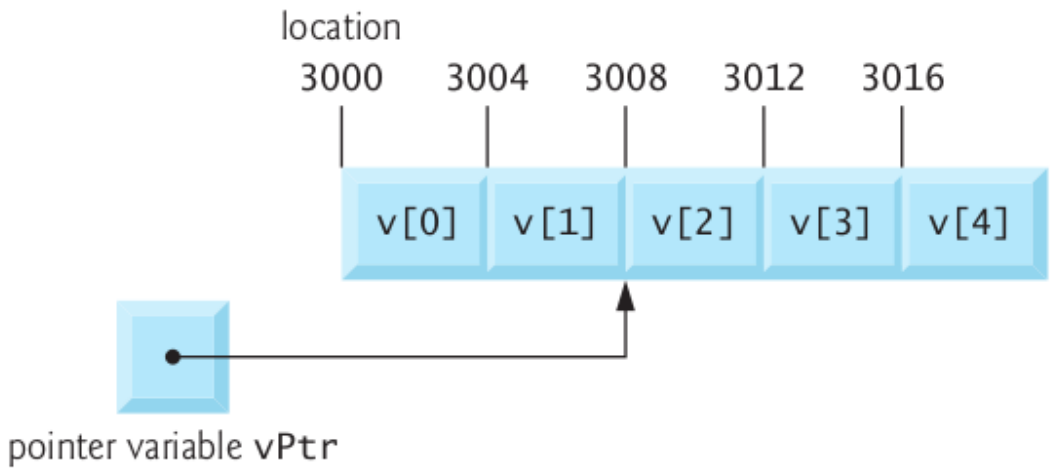
- İşaretçiler üzerinde aşağıdaki operatörler kullanılabilir
  - Artırma (++) ya da azaltma (--)
  - Bir işaretçiye tamsayı ekleme(+ ya da +=)
  - Bir işaretçiden bir tamsayı çıkarma(- ya da -=)
  - Bir işaretçiden başka işaretçiyi çıkarma
    - Ancak iki işaretçi de aynı diziyi gösterirse anlamlı
- Aşağıdaki senaryoyu düşünelim

```
vPtr = v;
vPtr = &v[0];
```



İşaretçi İfadeleri ve Aritmetiği

- `vPtr += 2;` işlemi **3008** adresini gösterir
- Geleneksel aritmetikte 3000 + 2 → 3002 yapar
- İşaretçi aritmetiğinde veri türü devreye girer, adres 2 sonraki konumu gösterecek şekilde değişir
- 3000 + 2 * 4 → 3008



İşaretçi İfadeleri ve Aritmetiği

- `vPtr`, 3016 değerine sahipse `vPtr -= 4;` ifadesi, işaretçiyi 3000 adresine(dizinin başı) çeker
- Artırma ve azaltma
  - `++vPtr;` ya da `vPtr++;` dizideki sonraki elemana
  - `--vPtr;` ya da `vPtr--;` dizideki önceki elemana geçer
- `vPtr`: 3000, `vPtr2`: 3008 adreslerinde olsun
  - `x = vPtr2 - vPtr;` işlemi sonucunda 2 değerini verir(8 değil)
- Aynı türden işaretçiler birbirlerine atanabilir
  - void türündeki işaretçi `void *` bunun **istisnasıdır**
- `void *` türündeki işaretçilerde referans kaldırma(`*`) yapılamaz
- `void *` türdeki işaretçiler genel işaretçi türüdür
- İşaretçiler eşitlik ve karşılaştırma operatörleri ile karşılaştırılabilir

Diziler ve İşaretçiler Arasındaki İlişki

- Diziler ve işaretçiler birbirleriyle yakından ilişkilidir ve birbirinin yerine kullanılabilir.
- Bir dizinin adı, dizinin başlangıç adresini gösteren sabit bir işaretçi olarak düşünülebilir.
- `int b[5]; int *bPtr;` tanımlarını düşünelim:
  - Dizi adı(yalnızca), dizinin ilk elemanının adresini verdiği için `bPtr = b;` ataması yapılabilir.
  - Bu işlem, `bPtr = &b[0];` ile aynı işleve sahiptir.
  - `b[3]` değeri, alternatif olarak `*(bPtr + 3)` ifadesi ile de elde edilebilir
    - bPtr adresinden 3 öteleme(offset) ile ilgili adrese ulaşılır,
    - Referans kaldırma operatörü(`*`) ile değer elde edilir
  - `&b[3]`, `bPtr + 3` şeklinde de ifade edilebilir
  - Dizi adı, işaretçi gibi kullanılabilir `*(b + 3)`
  - İşaretçiler ile `[]` operatörü kullanılabilir `bPtr[1]`
  - Dizi adı sabit olduğu için değiştirilemez, `b += 3` **geçersiz** bir işlemdir

Diziler ve İşaretçiler Arasındaki İlişki

fig07_14.c

```
1  // fig07_14.cpp
2  // Diziyle birlikte alt simge ve işaretçi gösterimlerini kullanma.
3  #include <stdio.h>
4  #define DIZI_BOYUTU 4
5
6  int main(void) {
7     int b[] = {10, 20, 30, 40}; // dizi b'yi oluştur ve başlat
8     int *bPtr = b; // bPtr'yi oluştur ve dizi b'ye işaret ettir
9     // dizi b'yi dizi alt simgesi gösterimiyle yazdır
10    puts("Dizi b, dizi alt simgesi gösterimiyle yazdırıldı:");
11    // dizi b boyunca döngü
12    for (size_t i = 0; i < DIZI_BOYUTU; ++i) {
13       printf("b[%zu] = %d\n", i, b[i]);
14    }
15    // dizi b'yi dizi adı ve işaretçi/ötelenme gösterimiyle yazdır
16    puts("\nİşaretçi/ötelenme gösterimi, işaretçinin dizi adı olduğu yer");
17    // dizi b boyunca döngü
18    for (size_t offset = 0; offset < DIZI_BOYUTU; ++offset) {
19       printf("*(b + %zu) = %d\n", offset, *(b + offset));
20    }
```

Diziler ve İşaretçiler Arasındaki İlişki

fig07_14.c

```
21    // dizi b'yi bPtr ve dizi alt simgesi gösterimiyle yazdır
22    puts("\nİşaretçi alt simgesi gösterimi");
23    // dizi b boyunca döngü
24    for (size_t i = 0; i < DIZI_BOYUTU; ++i) {
25       printf("bPtr[%zu] = %d\n", i, bPtr[i]);
26    }
27    // dizi b'yi bPtr ve işaretçi/ötelenme gösterimiyle yazdır
28    puts("\nİşaretçi/ötelenme gösterimi");
29    // dizi b boyunca döngü
30    for (size_t offset = 0; offset < DIZI_BOYUTU; ++offset) {
31       printf("*(bPtr + %zu) = %d\n", offset, *(bPtr + offset));
32    }
33 }
```

Diziler ve İşaretçiler Arasındaki İlişki

```
Dizi b, dizi alt simgesi gösterimiyle yazdırıldı:
b[0] = 10
b[1] = 20
b[2] = 30
b[3] = 40

İşaretçi/ötelenme gösterimi, işaretçinin dizi adı olduğu yer
*(b + 0) = 10
*(b + 1) = 20
*(b + 2) = 30
*(b + 3) = 40

İşaretçi alt simgesi gösterimi
bPtr[0] = 10
bPtr[1] = 20
bPtr[2] = 30
bPtr[3] = 40

İşaretçi/ötelenme gösterimi
*(bPtr + 0) = 10
*(bPtr + 1) = 20
*(bPtr + 2) = 30
*(bPtr + 3) = 40
```

Diziler ve İşaretçiler Kullanarak Dize(String) Kopyalama

fig07_15.c

```c
// fig07_15.c
// Dizi gösterimi ve işaretçi gösterimini kullanarak bir dizeyi kopyalama.
#include <stdio.h>
#define BOYUT 10

void kopyala1(char * const s1, const char * const s2); // prototip
void kopyala2(char *s1, const char *s2); // prototip

int main(void) {
   char string1[BOYUT]; // string1 dizisini oluştur
   char *string2 = "Merhaba"; // bir dizeye işaretçi oluştur

   kopyala1(string1, string2);
   printf("string1 = %s\n", string1);

   char string3[BOYUT]; // string3 dizisini oluştur
   char string4[] = "Güle Güle"; // bir dize içeren bir dizi oluştur

   kopyala2(string3, string4);
   printf("string3 = %s\n", string3);
```

Diziler ve İşaretçiler Kullanarak Dize(String) Kopyalama

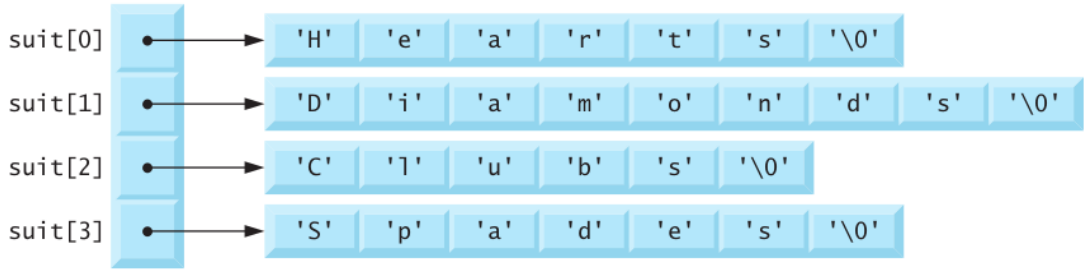
fig07_15.c

```
21 }
22
23 // s2'yi s1'e dizi gösterimi kullanarak kopyala
24 void kopyala1(char * const s1, const char * const s2) {
25    // diziler boyunca döngü
26    for (size_t i = 0; (s1[i] = s2[i]) != '\0'; ++i) {
27       ; // gövdede hiçbir şey yapma
28    }
29 }
30
31 // s2'yi s1'e işaretçi gösterimi kullanarak kopyala
32 void kopyala2(char *s1, const char *s2) {
33    // diziler boyunca döngü
34    for (; (*s1 = *s2) != '\0'; ++s1, ++s2) {
35       ; // gövdede hiçbir şey yapma
36    }
37 }
```

```
string1 = Merhaba
string3 = Güle Güle
```

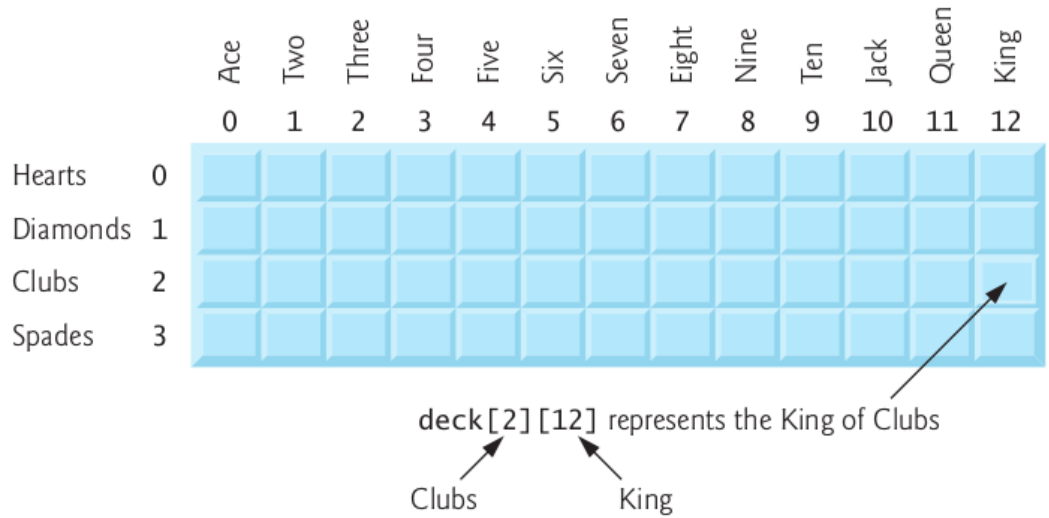
İşaretçi Dizileri

- Diziler içinde işaretçiler olabilir
- Sıklıkla kullanılan biçimi, dize(string) dizileridir
- `const char *suit[4] = {"Hearts", "Diamonds", "Clubs", "Spades"};`
- Dizinin her bir elemanı bir string'i(`char *`) gösterir



Rastgele Sayı Vaka Analizi

- Rastgele sayı üretimini kullanarak deste karıştırma ve dağıtma algoritması oluşturalım
- Daha sonra bu yöntem kart oyunları için kullanılabilir
- Sonraki bölümlerde daha optimize çalışan algoritmalar incelenecektir
- 52 kartın olduğu bir deste karıştırılarak sırayla kartlar çekilecektir
- Algoritma geliştirme için yukarıdan aşağıya, adım adım iyileştirme yaklaşımını kullanılacaktır
- Satırlar kart türünü, sütunlar kart değerini(yüz) ifade edecek



Rastgele Sayı Vaka Analizi

Desteyi Karıştırmak

- Destedeki(dizideki) tüm elemanlar sıfır(0) değeri ile başlatılır
- Rastgele bir satır(0-3) ve bir sütun seçilir(0-12)
- **1** değeri bu konuma yerleştirilir
- Bu işlem 2,3, . . . , 52 için tekrar edilir(eğer ilgili değer 0 ise)
- Eğer değer sıfırdan farklı ise rastgele başka bir konum seçilir
- Dizi doldukça yer bulmak zorlaşacaktır
- Karıştırılmış dizi tekrar karıştırılmaya kalkılırsa, sonsuz döngü oluşur

Rastgele Sayı Vaka Analizi I

Yukardan Aşağı Yaklaşım

- Yukarıdaki işlem basit
  - 52 kartı karıştır ve dağıt
- İlk iyileştirme
  - Tür dizisini başlat
  - Yüz dizisini başlat
  - Deste dizisini başlat
  - Desteyi karıştır
  - 52 kartı dağıt
- "Desteyi karıştır" aşağıdaki şekilde tanımlanabilir
  - Kart numarasını boş olan deste elemanına yerleştir
- "52 kartı dağıt" aşağıdaki şekilde tanımlanabilir
  - Kart numarasını deste dizisinde bul, türü ve yüzü yazdır

Rastgele Sayı Vaka Analizi II

Yukardan Aşağı Yaklaşım

- İkinci iyileştirme
  - Tür dizisini başlat
  - Yüz dizisini başlat
  - Deste dizisini başlat
  - 52 kartın her biri için
    - Kart numarasını destedeki rastgele seçilen, işgal edilmemiş bir konuma yerleştir
  - 52 kartın her biri için
    - Kart numarasını deste dizisinde bul ve kartın türünü ve yüzünü yazdır
- "Kart numarasını destedeki rastgele seçilen, işgal edilmemiş bir konuma yerleştir" işlemi:
  - Destede rastgele bir konum seç
  - Seçilen konum daha önce seçildiği sürece(dolu ise):
    - Destede rastgele bir konum seç
  - Kart numarasını seçilen konuma yerleştir

Rastgele Sayı Vaka Analizi III

Yukardan Aşağı Yaklaşım

- "Kart numarasını deste dizisinde bul ve kartın türünü ve yüzünü yazdır" işlemi:
  - Deste dizisinin her bir konumu için
    - Eğer konum kart numarasını içeriyorsa
      Kartın türünü ve yüzünü yazdır

Rastgele Sayı Vaka Analizi IV

Yukardan Aşağı Yaklaşım

- Geliştirmeleri yapunca üçüncü iyileştirme:
  - Tür dizisini başlat
  - Yüz dizisini başlat
  - Deste dizisini başlat
  - 52 kartın her biri için
    - Destede rastgele bir konum seç
    - Seçilen konum daha önce seçildiği sürece(dolu ise):
      Destede rastgele bir konum seç
    - Kart numarasını seçilen konuma yerleştir
  - 52 kartın her biri için
    - Destedeki her bir konum için
      Eğer konum istenen numarayı içeriyorsa
      Kartın türünü ve yüzünü yazdır

Rastgele Sayı Vaka Analizi

fig07_16.c

```c
// fig07_16.c
// Kartları karıştırma ve dağıtma.
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TUR 4
#define YUZLER 13
#define KARTLAR 52

// prototipler
void karistir(int deste[][YUZLER]);
void dagit(int deste[][YUZLER], const char *yuz[], const char *tur[]);

int main(void) {
   // deste dizisini başlat
   int deste[TUR][YUZLER] = {0};
   srand(time(NULL)); // rastgele sayı üretecini başlat
   karistir(deste); // desteyi karıştır
   // tur dizisini başlat
   const char *tur[TUR] = {"Kupa", "Karo", "Sinek", "Maca"};
   // yüz dizisini başlat
   const char *yuz[YUZLER] = {"As", "Iki", "Uc", "Dort", "Bes",
       "Alti", "Yedi", "Sekiz", "Dokuz", "On", "Vale", "Kiz", "Papaz"};
   dagit(deste, yuz, tur); // desteyi dağıt
}
```

Rastgele Sayı Vaka Analizi

fig07_16.c

```
28 // destedeki kartları karıştır
29 void karistir(int deste[][YUZLER]) {
30    // her kart için, destedeki bir yuva rastgele seç
31    for (size_t kart = 1; kart <= KARTLAR; ++kart) {
32       size_t satir = 0; // satır numarası
33       size_t sutun = 0; // sütun numarası
34
35       // boş yuva bulunana kadar yeni rastgele konum seç
36       do {
37          satir = rand() % TUR;
38          sutun = rand() % YUZLER;
39       } while(deste[satir][sutun] != 0);
40
41       deste[satir][sutun] = kart; // seçilen yuvaya kart numarasını yerleştir
42    }
43 }
```

Rastgele Sayı Vaka Analizi

fig07_16.c

```
45 // destedeki kartları dağıt
46 void dagit(int deste[][YUZLER], const char *yuz[], const char *tur[]) {
47    // her kartı dağıt
48    for (size_t kart = 1; kart <= KARTLAR; ++kart) {
49       // deste satırları boyunca döngü
50       for (size_t satir = 0; satir < TUR; ++satir) {
51          // geçerli satır için deste sütunları boyunca döngü
52          for (size_t sutun = 0; sutun < YUZLER; ++sutun) {
53             // yuva geçerli kartı içeriyorsa, kartı göster
54             if (deste[satir][sutun] == kart) {
55                printf("%5s %-5s  %c", tur[satir], yuz[sutun],
56                   kart % 4 == 0 ? '\n' : '\0'); // 4-sütunlu biçim
57             }
58          }
59       }
60    }
61 }
```

Rastgele Sayı Vaka Analizi

```
 Karo Kiz     Maca Bes     Karo Yedi   Sinek Alti
 Karo Sekiz   Maca Sekiz  Sinek Vale    Maca Vale
 Karo Papaz   Karo As     Sinek Dokuz   Kupa Kiz
 Kupa Iki    Sinek As      Kupa Sekiz  Sinek On
 Maca Papaz   Kupa Papaz   Kupa Dokuz   Karo Vale
 Karo Iki     Kupa Vale   Sinek Dort    Maca Kiz
 Maca Dokuz   Kupa As     Sinek Uc      Maca Uc
 Maca Yedi   Sinek Bes     Karo On      Kupa Alti
 Kupa On      Kupa Dort    Karo Uc      Maca As
 Maca On     Sinek Yedi    Karo Bes     Kupa Yedi
 Kupa Bes     Maca Alti   Sinek Papaz  Sinek Sekiz
Sinek Kiz    Sinek Iki     Karo Alti    Maca Dort
 Kupa Uc      Maca Iki     Karo Dokuz   Karo Dort
```

Fonksiyon İşaretçileri

fig07_17.c

```c
// fig07_17.c
// Çok amaçlı sıralama programı, fonksiyon işaretçileri kullanarak.
#include <stdio.h>
#define BOYUT 10

// prototipler
void kabarcikSiralama(int dizi[], size_t boyut, int (*karsilastir)(int a, int b));
int artan(int a, int b);
int azalan(int a, int b);

int main(void) {
   // sırasız dizi a'yı başlat
   int a[BOYUT] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };

   printf("%s", "Artana göre sıralamak için 1'e basın,\n"
           "Azalana göre sıralamak için 2'ye basın: ");
   int sira = 0;
   scanf("%d", &sira);

   puts("\nVeri öğeleri orijinal sırada");

   // orijinal diziyi yazdır
   for (size_t sayac = 0; sayac < BOYUT; ++sayac) {
      printf("%5d", a[sayac]);
   }
```

Fonksiyon İşaretçileri

fig07_17.c

```
27    // diziyi artan sıraya göre sırala; artan sıralama düzenini belirtmek için
28    // argüman olarak artan fonksiyonunu geç
29    if (sira == 1) {
30       kabarcikSiralama(a, BOYUT, artan);
31       puts("\nVeri öğeleri artan sırada");
32    }
33    else { // azalan fonksiyonunu geç
34       kabarcikSiralama(a, BOYUT, azalan);
35       puts("\nVeri öğeleri azalan sırada");
36    }
37
38    // sıralanmış diziyi yazdır
39    for (size_t sayac = 0; sayac < BOYUT; ++sayac) {
40       printf("%5d", a[sayac]);
41    }
42
43    puts("\n");
44 }
```

Fonksiyon İşaretçileri

fig07_17.c

```
46 // çok amaçlı kabarcık sıralama; karşılaştır parametresi, sıralama düzenini belirleyen
47 // karşılaştırma fonksiyonuna işaret eden bir işaretçidir
48 void kabarcikSiralama(int dizi[], size_t boyut, int (*karsilastir)(int a, int b)) {
49    void degistir(int *eleman1Ptr, int *eleman2Ptr); // prototip
50
51    // geçişleri kontrol eden döngü
52    for (int gecis = 1; gecis < boyut; ++gecis) {
53       // her geçişteki karşılaştırma sayısını kontrol eden döngü
54       for (size_t sayac = 0; sayac < boyut - 1; ++sayac) {
55          // bitişik elemanlar sırasızsa, yer değiştir
56          if ((*karsilastir)(dizi[sayac], dizi[sayac + 1])) {
57             degistir(&dizi[sayac], &dizi[sayac + 1]);
58          }
59       }
60    }
61 }
```

Fonksiyon İşaretçileri

fig07_17.c

```
63 // element1Ptr ve element2Ptr'nin işaret ettiği bellek konumlarındaki değerleri değiştir
64 void degistir(int *eleman1Ptr, int *eleman2Ptr) {
65    int tut = *eleman1Ptr;
66    *eleman1Ptr = *eleman2Ptr;
67    *eleman2Ptr = tut;
68 }
69
70 // elemanların artan sıralama için sırasız olup olmadığını belirle
71 int artan(int a, int b) {
72    return b < a; // b, a'dan küçükse yer değiştirmeli
73 }
74
75 // elemanların azalan sıralama için sırasız olup olmadığını belirle
76 int azalan(int a, int b) {
77    return b > a; // b, a'dan büyükse yer değiştirmeli
78 }
```

Fonksiyon İşaretçileri

fig07_18.c: Menü örneği

```c
// fig07_18.c
// Fonksiyon işaretçilerinin bir dizisini gösterme.
#include <stdio.h>

// prototipler
void fonksiyon1(int a);
void fonksiyon2(int b);
void fonksiyon3(int c);

int main(void) {
   // her biri bir int argüman alan ve void döndüren 3 fonksiyon için
   // fonksiyon işaretçileri dizisini başlat
   void (*f[3])(int) = {fonksiyon1, fonksiyon2, fonksiyon3};

   printf("%s", "0 ile 2 arasında bir sayı girin, bitirmek için 3: ");
   int secim = 0;
   scanf("%d", &secim);

   // kullanıcının seçimini işle
   while (secim >= 0 && secim < 3) {
      // f dizisindeki seçim konumundaki fonksiyonu çağır ve
      // argüman olarak seçim'i geç
      (*f[secim])(secim);

      printf("%s", "0 ile 2 arasında bir sayı girin, bitirmek için 3: ");
      scanf("%d", &secim);
   }
```

Fonksiyon İşaretçileri

fig07_18.c: Menü örneği

```
29    puts("Program çalışması tamamlandı.");
30 }
31
32 void fonksiyon1(int a) {
33    printf("Siz %d girdiniz, bu yüzden fonksiyon1 çağrıldı\n\n", a);
34 }
35
36 void fonksiyon2(int b) {
37    printf("Siz %d girdiniz, bu yüzden fonksiyon2 çağrıldı\n\n", b);
38 }
39
40 void fonksiyon3(int c) {
41    printf("Siz %d girdiniz, bu yüzden fonksiyon3 çağrıldı\n\n", c);
42 }
```